DESE71003 – Sensing and Internet of Things

Imperial College London
Dyson School of Design Engineering

Using temperature and humidity to build the world's worst intruder alarm system

Author:
Francesco Rastelli
02310086

Code & Date URL:
https://github.com/TheLegoCraft/SIoT.git

Coursework Video URL:
https://youtu.be/MTpRo2CGLB8

Date: December 10, 2024

Abstract

This project investigates whether indoor humidity and indoor temperature compared to the outdoor temperature can be used to detect intruders in a room and trigger alerts for them. The project uses a Raspberry Pi 3B+ to compute and store the data, a DHT11 humidity and temperature sensor, and finally Open Meteo API to collect external temperature at 15-minute intervals. By analysing the data collected, we could then work out if 0, 1 or 2+ people were present in the room in the last hour. Intruder presence would send an email alert to the phone, while the data would be stored locally and could be accessed remotely via a Flask REST API.

Introduction

You may be asking yourself how I got to the point of using temperature and humidity as a security system, and for that, I wouldn't blame you. To be honest, the start of this project was not aimed to be built towards a security system at all. Instead, the original idea focused on using the indoor humidity and difference in temperature between the inside and outside of my bedroom window to work out when that window would build condensation on it, and so activate a dehumidifier to dry it. However, as I was collecting data, two realisations came to me: the only data I would actually need to collect for this is the humidity in the room, and what the data was actually showing was how many people and more importantly, when people were in the room. As such, I began exploring how I could use the data I collected to build a system which would alert my phone whenever 1 or more people were present in the room.

While normal, boring, security systems might use a camera, or a motion sensor, or even lasers, to detect presence in a room, these all have one major issue: they require the person to be in direct line of sight. This major hurdle starts impacting the systems as soon as you want the monitoring system to be camouflaged and hidden from view. However, the ambient room humidity and temperature remain more or less constant throughout a room, meaning that a security system which uses it to detect how many people are present in a room could have some serious advantages if built upon properly.

While the objectives may have shifted from the start, when looking at a security system, a few important goals came up:

1. The system should be able to detect the difference between 0, 1 and 2+ people in the room.
2. The system should alert the user when there is 1 or 2+ people present in the room.
3. Record and store the data so a user can know previous intruder presence.

Sensing

Data Sources

With the initial aim being to work out when my windows built up condensation and fogged up, I decided that the data I would need to collect is indoor humidity, indoor temperature and outdoor temperature. This is because the window generally builds up condensation when there is high humidity in the room (caused by people breathing and sweating), and a high difference in the temperature inside and outside causing the window to be really cold, allowing water vapour in the air to condensate on it. However, as I explain in the results, when analysing the data I found out that I was actually recording was the presence of someone in the room, and I could also work out how many people were present through the humidity.

The first challenge I had to tackle was deciding which computer board to use to run the date collection, store it, and allow me to access it remotely. The more obvious choice for this was a Raspberry Pi 3B+, as its ability to easily connect to the internet, its integration with Python, and the ability to connect external sensors through GPIO pins, made it perfect for the task. This allowed me to use it as a local server which I could remotely connect to through the Flask REST API.

When looking for a reasonable module to connect to it to collect the indoor temperature and humidity data, I ended deciding on the

DHT11 Digital Temperature and Humidity Sensor as I could power it off the Raspberry Pi itself and read the data through one of the GPIO pins. Finally, the last set of data I would need to collect is outdoor temperature; to do so, I decided to use the Open Meteo API which allowed me to request the exact outdoor temperature at the latitude and longitude of the house.



Figure 1. Picture of the prototype

When I was running the collection, I decided an appropriate rate of collection for the data was 15 minutes, as the rate at which humidity and temperature would rise in a room was slow enough that a faster detection speed would not be needed. While running, I had a small screen set up to ensure that if and when an error or failure which I hadn't accounted for happened, I could see it and react to it quickly.

Storing the data

When it came to storing this data, I used .txt files stored locally on the Raspberry Pi that would be saved in a /Log folder and then pushed to GitHub when needed. Every time the program was started, a new log file was created, and 4 sets of data would be appended to it every time a new measurement was made. Each line log had the following structure: *"2025-11-28T17:19:41.778891, 81, 20.7, 11.8"* where the data is structured as: date & time, humidity, inside temperature, outside temperature.

This would act as a local server, which could then be accessed remotely either through a Raspberry Shell, or through the Flask REST API. This API is a program that essentially used the Raspberry Pi as a small web server; from this, I can then define URL endpoints which read the log files, covert the data into JSON and allow someone to read the data when someone does a GET request.
In the program, there are 3 kinds of GET request. First, you have the simplest API command:
"curl http://192.168.1.131:5000/"
which outputs the structure of the possible JSON's that can be called as:
"World's Worst Security System - Data API, endpoints: /latest, /history" which is then used by the user to see the options available to them. If a user adds *"/latest"* to the command above, they will receive the latest reading from the log files; while, if they add *"/history"* they will receive the whole set of logged data that is currently stored on the Raspberry Pi.

Overall system architecture

The overall system architecture can be broken down into: data sources in yellow, key data handling functions in orange, the local data storage in light blue, Flask REST API in purple, and finally the external "intruders" and user in white. The Raspberry Pi (red) in the middle encompasses the majority of the key processes.

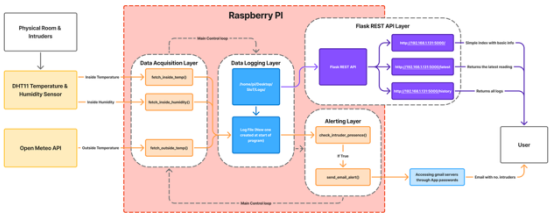


Figure 2. Data and system architecture

Challenges faced

One of the first challenges I faced was with the DHT11 sensor failing to read either humidity or temperature when pinged. This was combined with an issue where the Open Meteo API would sometimes fail, such as when the servers would crash. The fix for both of these (which would also prove to be helpful when recording the data in the logs) was to add *RuntimeError* checks to skip that data point reading in the program.

```
# Function to get inside temperature
def fetch_inside_temp(dht):
    # Attemps to get humidity and return it
    try:
        indoor_temp = dht.temperature
        return indoor_temp
    except RuntimeError as e:
        print("indoor_temp ERROR") # Will show the error on the terminal
        return None
```

Figure 3. Example of RuntimeError check

The next big issue came when trying to convert the log files from .txt back into their variables so old data could be read and analysed. This often resulted in a *ValueError* or a *TypeError*, which could be solved by adding "*.float()"* for the humidity and temperature values, while for the time stamp, using "*datetime.fromisoformat()"* helped to prevent these errors. Adding an *"except()"* check also helped to ensure that no errors actually sneaked through.

```
# Coverts the strings to their appropriate format
try:
    # Converts the strings to their appropriate format
    timestamp = datetime.fromisoformat(timestamp)
    indoor_humidity = float(indoor_humidity)
    indoor_temperature = float(indoor_temperature)
    outdoor_temperature = float(outdoor_temperature)
except (ValueError, TypeError):
    # If conversion fails, skip this line
    return None
```

Figure 4. ValueError and TypeError check

The last major challenge I had to deal with was that the Raspberry Pi would crash when starting the server. I suspected this was due to a RAM issue, as running a local server off the Raspberry Pi was a very resource intensive, however, I was able to solve this by using part of the storage in the SD card as overflow RAM.

Results

The graphs below show the data collected over a week inside my bedroom. The blue lines show humidity in the room across all 3 graphs, while the red line shows the inside, outside and difference in temperature respectively.

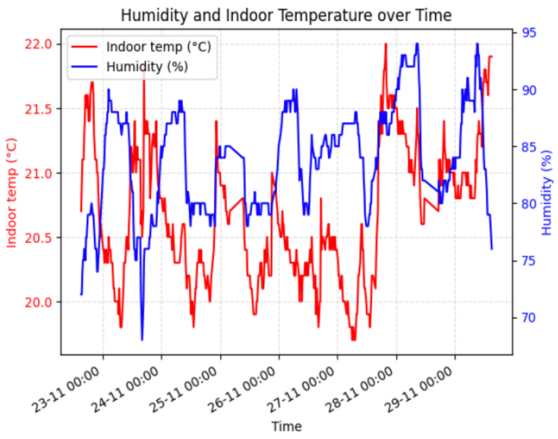


Figure 5. Humidity and indoor temperature

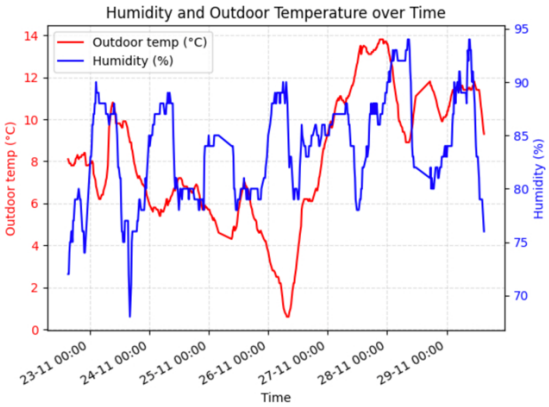


Figure 6. Humidity and outdoor temperature

The last graph has been labelled to show the presence of someone in the room:

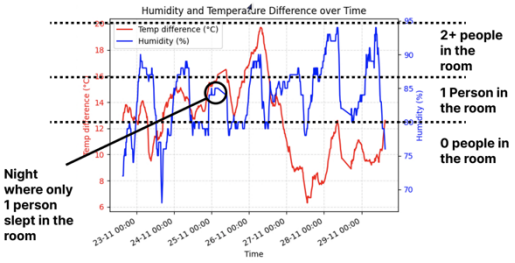


Figure 7. Humidity and temperature change

When analysing the graph above, I noticed a particular pattern which then influenced the decision to make "the world's worst security system". The first 2 nights had 2 people sleeping in the room, and humidity reached around 87 to 92% during these nights. However, on the 3rd night, when only 1 person slept in the room, humidity did not go above 85%. This was repeated on the 5th night where humidity only rose above 85% when I arrived back after 4am and so only 2 people were present after that. Meanwhile, during the day where both I and my roommate were working, the humidity remains stable, showing that it only increases when someone is present in the room. The sharp drops in the humidity in the mornings, instead, come from opening the bedroom window as soon as 1 person wakes up. When it comes to temperature, ambient temperature in the room seems to increase when someone is present, and only sharply drop when the window is open, however, the exact temperature does not correlate to someone being present in the room, only normally if the temperature difference is increasing.

Internet of Things

The integration of IOT into this product involved 3 key things. Firstly, the Open Meteo API which I used as a data source to look at the outside weather data. Secondly, sending an alert to the user if 1 or more people were present in the room. And thirdly, allowing a user to remotely access the data through the Flask Rest API.

In this section, the focus was on alerting a user of when an intruder was present in the room. Using humidity and temperature to check for intruder presence involved reading the last 4 datapoints in the latest log (equivalent to 1 hour), with a check in place to ensure that if 4 data points had not been taken, the file wouldn't crash. In both cases, for the intruder alert to trigger, firstly, the temperature difference should be increasing. Then, once the temperature difference has been confirmed to be increasing, we check what range the humidity is in. If average humidity over the 4 data points is between 80 and 87%, we can safely estimate that 1 person is present in the room, while if humidity is above 85%, we can assume that at least 2 people are present in the room. As such, the only option left is when humidity is below 80%, which is representative of when no one is in the room.

```
# Them, we check the last hour of data (last 4 data points in log).
if temp_increase > 0:
    if 80 <= humidity_average <= 87:
        return 1
    elif humidity_average >= 87:
        return 2
    else:
        return 0
else:
    return 0
```

Figure 8. Intruder check for the room

The original plan to alert was the user was to use an SMS emulator API, such as Twilio, to send an SMS directly to their phone. However, as there was no free option to do this, the best solution I found was to go through email and alert the user on that. Integration of email into a Python script involved 3 main steps. Firstly, I had to go to a Gmail account and set up "App passwords" to allow my Raspberry Pi and Python to access and send emails through a special 16-digit code. Next, to ensure safety and privacy, I stored these directly on *".bashrc"* (which is equivalent to the Raspberry Pi's os environment) which allowed me to have global values that could be accessed in python through *"os.environ["NAME"]"* without having them be directly visible in the python script. Lastly, checking that the last email hadn't been sent in the previous hour, I was able to email an alert to my university email when the check proved true with the subject: "RASPBERRY PI HAS DETECTED INTRUDERS!" and the body structured as *"{number_intruder} intruder(s) have been detected in the room at {datetime.now()}.*

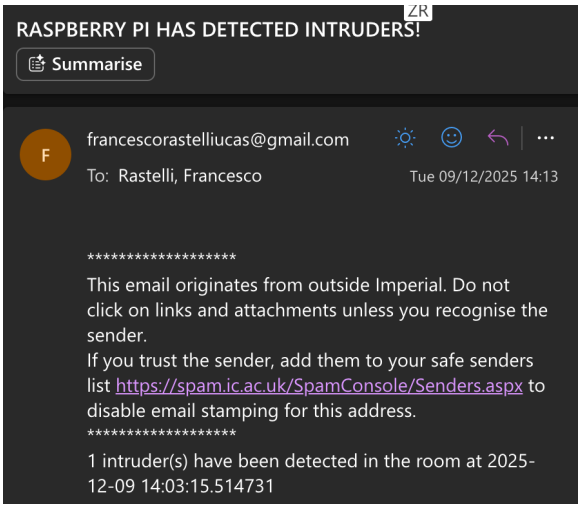


Figure 9. Example of an intruder alert

Challenges faced

As this was the first time I was editing *".bashrc"* through nano, I struggled to adapt to it at the start. As such, I did not realise that there was a difference in the type of quotes that you could type (in my case I originally had smart quotes when I was meant to have plain ASCII quotes). This resulted in a *"KeyError"* when trying to send an email as when it called the email address from the os, instead of receiving *"francescorastelliucas@gmail.com",* it received: *"francescorastelliucas@gmail.com" export ALERT_EMAIL_PASSWORD = your_app_password_here" PASS: None".* This both bricked the python program but also revealed the error quickly as it was fairly evident what had happened, while teaching me an important lesson.

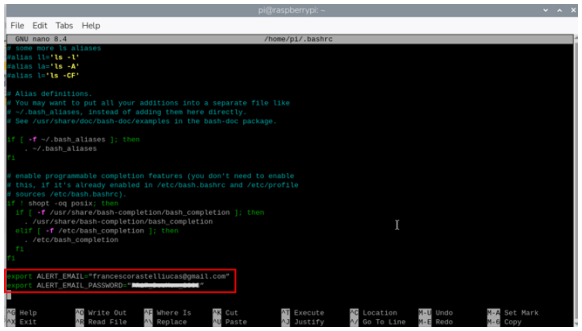


Figure 10. Editing the .bashrc environment

Another key issue I faced was with combining the data collection and processing with the Flask REST API. When the Flask REST API runs, it blocks the rest of the program for running, so there were two solutions to fix this. The first and easier solution was to stop and restart the API within the loop, however, this would create two big issues: if a user sent a GET request while the API wasn't running, they would get an error back, and more importantly, as the API takes time to boot up, it would slow the rest of the process down. As such, the second option, which was to run the API in a separate thread, proved to be the best; this allowed the API to run in the background of the program where any user could call it, while the main logic could continue running in the foreground.

```
def start_api():
    # host='0.0.0.0' makes the server visible on your LAN.
    # port=5000 is the default Flask port.
    app.run(host="0.0.0.0", port=5000)

# Run the Flask API in a seperate thread
api_thread = Thread(target=start_api)
api_thread.daemon = True
api_thread.start()
```

Figure 11. Additional thread for the API

Analysis of the product

Performance

When looking at the performance of the program, the main issue arises from the computational limits of the Raspberry Pi 3B+. This is as the PI is trying to read data, call an external API, run a Flask server and send emails simultaneously. While the sampling rate of 15 minutes does help to alleviate this issue, it could also result in a detection latency; although it should be noted that the humidity and temperature in a room do take a while to rise, so even this issue is minimal.

If I was to redo this again, I would use a newer Raspberry Pi board such as the 5, as it has both more RAM, allowing me to avoid having to use the SD card for overflow, and a faster CPU, so I don't have to overclock it for it to be able to run both the API and the data collection at the same time.

Scalability

This system is not built for scalability and so faces a few issues at a large scale. Firstly, there are performance issue with the Flask API, as the Pi's limited CPU and RAM will constrain how many GET requests can be answered at once. Secondly, the use of *"/history"* with no limits means that after long periods of data recording, too much data will have been stored for the program to handle, potentially causing massive latency issues or even overflow errors. The last major issue is that the email alerts are hard coded to a single recipient. As such, if I was to improve the code, I will give the user the ability to input their own email into the code and add limits to the number of lines which are sent back to the user. A reasonable limit would be 672 lines, as that is the maximum number of data points that could be collected in a week (7*24*4=672).

Complexity

While the core logic is quite simple, the complexity of the program comes from the use of multiple APIs, the use of *".bashrc"* to have hidden and private variables, but also the ability to receive email alerts by connecting to the Gmail SMTP. The complexity of setting all this up would mean that the average user might struggle to do so correctly without lots of trial and error, much like how I had to do.

Security & Privacy

There are some massive issues with security and privacy. Firstly, the data would be accessible by anyone on the same Wi-Fi or LAN connection, as the Flask server has no authentication, encryption or access control. This could result in misuse of the data, as the program could be tracking the homeowner's presence in a specific room, such as the bedroom, and this information could be used for malicious reasons.

However, some measures were taken to ensure safety such as using environment variables in *".bashrc"* which reduces the risk of them being accidentally published, such as through GitHub. Furthermore, the only information which leaves the local network is the email alerts, meaning that the data is secure within that network itself.

Conclusion

Overall, the system was able to reliably distinguish between 0, 1 and 2+ people in this specific period through humidity and change in temperature. However, it should be noted that more whether data should be implemented to account for the ambient humidity in the air, as this can shift from season to season, meaning that while my prototype might be working now, if it was summer, it would most likely fail.

It was also able to alert a user when there were 1 or 2+ people present in the room through email, without any issues arising. This means it was fully able to fulfils the second goal, with the only minor improvement being the recipient email not being hard coded.

Finally, the data was recorded and stored in an appropriate manner. However, a major improvement would be having the data be stored on the cloud, such as on an AWS server; this would both reduce the load for the Raspberry Pi and would also allow any user to access the data without having to be connected to the same LAN.

While this system may not be as reliable or quick as a camera, a motion sensor or a laser, it still proves to be an interesting alternative for security which, if explored in more detail, could prove to be better than the standard in very, very specific cases.

GAI

GAI was used to help with the coding and debugging. However, please note that all the code and comments were rewritten by myself and the GAI was only used to aid me in understanding what the code meant and which functions to use. I also used it when summarising the information to write the abstract, but again that was rewritten in my own words.